

The Foundation Model Handbook

*Architecture, Pretraining, Post-training, Reinforcement Learning, Inference,
and Systems*

Author Name

Version 0.1 · 2026

Preface

This is a living handbook for studying Foundation Models. Each chapter will be added only after the underlying ideas have been learned, checked against primary sources, and organized into a durable explanation.

Contents

Preface	2
Part I — Foundations	5
Chapter 1 Tokenization and Input Representations	5
1.1 From text to a discrete sequence	5
1.2 Vocabulary design and reserved symbols	6
1.3 Byte Pair Encoding	6
1.4 Embedding lookup	7
1.5 Positions and the initial model state	8
1.6 Token counts as an engineering quantity	8
1.7 Implementation contract	8
Chapter 2 Transformer Architecture	9
2.1 From token IDs to hidden states	9
2.2 The decoder-only stack	10
2.3 Anatomy of a Transformer block	10
2.4 Pre-Norm and Post-Norm	11
2.5 The residual stream	12
2.6 From final hidden states to logits	12
2.7 Tensor shapes through the model	12
2.8 A computational view	13
2.9 Chapter summary	13
Chapter 3 Attention and Position Encoding	14
Chapter 4 Feed-Forward Networks, Normalization, and Residuals	14
Part II — Pretraining	15
Chapter 5 Language Modeling Objectives	15
Chapter 6 Pretraining Data	15
Chapter 7 Optimization and Training Dynamics	15
Chapter 8 Scaling, Compute, and Training Stability	15
Part III — Post-training and Adaptation	16
Chapter 9 Supervised Fine-Tuning and Preference Data	16
Chapter 10 Reinforcement Learning Foundations	16
Chapter 11 RLHF and PPO	16
Chapter 12 DPO, GRPO, and Reasoning RL	16
Chapter 13 Parameter-Efficient Fine-Tuning	16
Part IV — Inference	18
Chapter 14 Autoregressive Decoding and KV Cache	18
Chapter 15 Efficient Inference	18
Chapter 16 LLM Serving	18
Part V — LLM Systems	19
Chapter 17 GPU Memory, FLOPs, and Performance	19
Chapter 18 Distributed Training and Parallelism	19
Chapter 19 Training Infrastructure and Fault Recovery	19
Part VI — Evaluation and Applications	20
Chapter 20 Foundation Model Evaluation	20

Chapter 21 RAG, Tool Calling, and Agent Systems 20

Part I — Foundations

Chapter 1

Tokenization and Input Representations

Text reaches a Foundation Model through a deliberately narrow interface. The model does not receive characters, words, or bytes as linguistic objects; it receives a finite sequence of integer identifiers. That reduction is easy to overlook because tokenization is usually hidden behind a library call. It nevertheless fixes the model’s input alphabet, determines how much of a context window a document occupies, and allocates a substantial parameter table. A tokenizer is therefore part of the model specification, not merely preprocessing.

This chapter develops the input interface used throughout the handbook. Its immediate goal is to make the sequence entering a decoder-only Transformer mathematically explicit. Its longer-term purpose is practical: later chapters will count compute in tokens, form training batches from token sequences, and reuse the same identifiers in the output softmax. The reader should leave able to implement a small Byte Pair Encoding (BPE) tokenizer, inspect its behavior on text, and account for the tensors created before the first Transformer block.

1.1 From text to a discrete sequence

Let $\mathcal{S}$ denote the set of text strings after a chosen character encoding, and let $\mathcal{V}$ be a finite vocabulary. A tokenizer is a deterministic map from an input string to a finite sequence of vocabulary elements:

$$\tau : \mathcal{S} \rightarrow \mathcal{V}^*. \quad (1)$$

The star denotes the set of all finite sequences. In an implementation, each vocabulary element is assigned an integer identifier, so the output of τ is more conveniently written as

$$x = (x_1, \dots, x_T), \quad x_t \in \{0, \dots, |\mathcal{V}| - 1\}. \quad (2)$$

Here T is the tokenized length of the particular input, not a property of the vocabulary. The distinction matters. Two strings with similar character counts can have very different values of T , and the computational cost of a Transformer is governed primarily by token positions rather than character positions.

Definition 1: Tokenizer. A tokenizer consists of a finite vocabulary $\mathcal{V}$, a convention for converting text into an initial sequence of atomic symbols, a segmentation procedure that returns elements of $\mathcal{V}$, and an inverse decoding convention on the subset of sequences that it regards as valid.

The definition includes more than a merge list. Normalization, whitespace handling, the choice between Unicode characters and bytes, and the treatment of reserved symbols all affect the mapping

in Equation 1. A tokenizer need not be reversible over arbitrary input strings: a normalizer may intentionally collapse distinctions. It should, however, make its contract explicit. In a training system, the same contract must be used for corpus preparation, training examples, evaluation prompts, and generation-time decoding. Otherwise the model is trained and evaluated on different discrete languages.

The vocabulary is usually much smaller than the space of possible strings. A word-level vocabulary makes each frequent word inexpensive in sequence length, but it either assigns rare forms to an unknown symbol or grows without bound. A character-level vocabulary avoids most out-of-vocabulary failures, but represents common words by long sequences. Subword tokenization occupies the useful middle ground: common fragments may become single tokens, while uncommon strings are composed from smaller pieces. Sennrich, Haddow, and Birch adapted BPE to this open-vocabulary setting and showed that subword units can replace a fixed-vocabulary back-off mechanism in neural translation [1].

1.2 Vocabulary design and reserved symbols

A vocabulary contains ordinary text pieces and a small set of symbols with control semantics. Typical examples include a beginning-of-sequence marker, an end-of-sequence marker, padding, and reserved delimiters for message roles or document boundaries. These are not decorations around the text. They are token IDs whose embeddings are learned in exactly the same way as ordinary token embeddings, but whose positions carry a convention established by the training data and runtime.

The crucial design rule is that a special token must be handled as an atomic unit before ordinary segmentation. If a chat delimiter is sometimes decomposed into text pieces and sometimes inserted as one reserved ID, the model sees two incompatible representations of the same structural event. Conversely, a token that is reserved but never appears in training has no learned semantics merely because it has a vocabulary entry. Tokenizer configuration and data formatting must therefore be versioned together.

Vocabulary size also creates a direct parameter trade-off. A larger vocabulary can shorten sequences by storing more frequent substrings, but it enlarges both the input embedding table and, in many decoder-only models, the output classifier. It also changes the empirical distribution of targets: a very common token receives many updates, while a rare token receives few. There is no vocabulary size that is independently optimal; the appropriate choice depends on corpus coverage, languages, model width, and the intended context length.

1.3 Byte Pair Encoding

BPE originated as a compression algorithm that repeatedly replaces frequent adjacent symbol pairs with newly introduced symbols [2]. Its appeal for neural text processing is not compression alone. The same procedure constructs a vocabulary that can represent frequent strings compactly while retaining a base alphabet for rare strings. Modern BPE tokenizers differ in details, but the merge construction is sufficiently simple to derive directly.

Consider a corpus represented as sequences over an initial alphabet $\mathcal{V}_0$. The alphabet may be characters, bytes, or a reversible encoding of bytes. At merge step m , let $c_{m(a,b)}$ count occurrences of the adjacent pair (a, b) in the current representation. BPE selects a most frequent pair,

$$(a_m, b_m) = \arg \max_{a,b} c_{m(a,b)}, \quad (3)$$

introduces a new symbol u_m denoting the concatenation of a_m and b_m , and replaces the selected pair according to a fixed overlap convention. After M merges, the learned vocabulary is the base vocabulary together with the introduced symbols:

$$\mathcal{V}_M = \mathcal{V}_0 \cup \{u_1, \dots, u_M\}. \quad (4)$$

The recurrence exposes the central inductive bias. BPE does not discover linguistic morphemes by definition; it merges pairs that are frequent under its current representation. A frequent substring can become one token even if it is not a word, while a morphologically meaningful but rare substring may remain split. This is often appropriate for language modeling, whose immediate objective is to assign probabilities to symbol sequences, but it prevents us from treating token boundaries as a ground-truth linguistic analysis.

Algorithm 1: BPE vocabulary training. Start from an initial corpus representation over $\mathcal{V}_0$. At each step, count adjacent pairs, choose a pair with maximum count under a deterministic tie rule, introduce its concatenated symbol, and replace its eligible occurrences throughout the corpus. Record the ordered merge. Stop after the prescribed number of merges or when no pair is eligible.

Training the merge vocabulary and encoding a new string are separate procedures. Training estimates which pairs deserve vocabulary entries from a corpus. Encoding begins from the base symbols of a new string and applies only merges recorded during training, in their learned priority order. Implementations commonly store a rank for each merge and repeatedly select the best available adjacent pair. The rank order, base-symbol construction, and pre-tokenization policy are all part of the tokenizer artifact; changing any of them changes the IDs in Equation 2.

Byte-level BPE makes the base alphabet cover bytes rather than an application-specific set of characters. Provided the byte encoding itself is reversible, every input byte sequence can then be represented without an unknown token; rare text simply falls back to more base symbols. GPT-2 used a byte-level version of BPE, illustrating how this choice supplies a fixed vocabulary while retaining coverage of arbitrary text bytes [3]. By contrast, SentencePiece emphasizes training subword models directly from raw sentences, avoiding a language-specific requirement for pre-tokenized word boundaries [4]. These choices are implementation conventions, not interchangeable defaults: they influence whitespace behavior, multilingual coverage, and the reproducibility of a dataset pipeline.

1.4 Embedding lookup

The output of a tokenizer becomes useful to a neural network through an embedding table. Let d be the model width and let

$$E \in \mathbb{R}^{|\mathcal{V}| \times d} \quad (5)$$

be a trainable matrix. The vector associated with token ID x_t is the corresponding row,

$$e_t = E[x_t] \in \mathbb{R}^d. \quad (6)$$

This operation is a row lookup, although automatic-differentiation systems often implement it as multiplication by a one-hot vector. If q_t is the one-hot representation of x_t , then $e_t = q_t^T E$. The lookup view is operationally more useful: only the rows selected by a batch receive direct embedding gradients. A vocabulary item that appears rarely receives few such updates, while a control token inserted into every sequence may receive many.

For a batch of B padded sequences of maximum length T , the token-ID tensor has shape $B \times T$ and the embedding output has shape $B \times T \times d$. The embedding table contributes $|\mathcal{V}|d$ trainable

scalar parameters before any positional representation or Transformer layer is considered. This count is elementary, yet it explains why tokenization choices cannot be separated from model budgeting.

1.5 Positions and the initial model state

The embedding table associates a vector with a token identity, not with a location in a sequence. The word piece for a name has the same lookup vector whether it occurs first or last. A Transformer consequently needs an additional source of positional information. In the original Transformer formulation, position-dependent vectors are added to token embeddings before the stacked attention and feed-forward layers [5]. We write the initial state at position t as

$$h_t^0 = e_t + p_t, \quad (7)$$

where $p_t \in R^d$ is a positional representation. It may be a learned absolute embedding, a deterministic vector, or part of a later attention computation. The architectural variants and their extrapolation behavior belong to Chapter 3; the present point is narrower. Tokenization fixes the positions to which any positional scheme is applied. Altering a tokenizer can therefore change both the length and the positional geometry of every training example, even when the underlying text is unchanged.

Some implementations share the input embedding matrix with the output projection used to score the next token. Weight tying reduces parameters and links the geometry of input and output symbols, but it is an architectural option rather than a tokenizer requirement. The original Transformer described shared embedding and pre-softmax weights in some settings [5]. Later chapters will distinguish this representation layer from the language-model objective imposed on its outputs.

1.6 Token counts as an engineering quantity

For a corpus $\mathcal{D}$, define its token count under tokenizer τ by

$$N_{\tau(\mathcal{D})} = \sum_{s \in \mathcal{D}} |\tau(s)|. \quad (8)$$

This quantity recurs throughout the handbook. Pretraining budgets are normally reported in tokens, batches are assembled in token positions, and the cost of processing a fixed document collection varies with $N_{\tau(\mathcal{D})}$. A tokenizer that splits a corpus into longer sequences consumes more context positions and can increase the attention work associated with a training example. It may nevertheless be preferable if its base representation improves coverage or avoids brittle text normalization. The correct comparison is therefore not a slogan such as “fewer tokens is better,” but a joint assessment of coverage, modeling behavior, parameter cost, and systems cost.

The same caution applies to evaluation. Perplexity and average negative log-likelihood are defined per model token, so their numerical values are meaningful only relative to a specified tokenizer and normalization pipeline. Comparing token-level losses across incompatible tokenizations can confound changes in modeling quality with changes in the units being predicted. Chapter 20 returns to this issue when defining language-model evaluation.

1.7 Implementation contract

A minimal tokenizer implementation should expose four testable operations: encode text to IDs, decode valid IDs to text, identify reserved tokens, and serialize the complete vocabulary plus segmentation rules. Round-trip tests should be performed on representative text, including whitespace, non-ASCII characters, delimiters, and malformed inputs where the contract specifies behavior. The

test does not prove that a tokenizer is linguistically desirable; it establishes that the discrete interface is stable.

For BPE, a small implementation is especially instructive. Begin with a corpus of short strings, represent each string as a sequence of base symbols, count adjacent pairs, apply the merge rule in Equation 3, and store the resulting ranks. Then encode held-out strings using those ranks. The exercise makes two facts concrete: merge training is corpus dependent, and inference-time segmentation is constrained by a fixed learned artifact. Both facts become important when training data, checkpoints, or evaluation sets are revised.

Tokenization is one of the first irreversible modeling decisions in a language-model pipeline. It defines the symbols that later chapters will embed, predict, cache, shard, and evaluate. Chapter 2 takes the resulting sequence of IDs as the input to a complete decoder-only computation graph; Chapter 5 later specifies the causal prediction objective. Before proceeding, the reader should be able to trace a string through τ , verify the resulting IDs and embeddings, and explain which tokenizer choices alter the token budget in Equation 8.

Chapter 2

Transformer Architecture

A decoder-only Transformer is often introduced as a stack of Self-Attention layers. That description is useful but incomplete. A language model has a stable external contract: it receives a tensor of token IDs and produces one vocabulary-sized vector of logits at every position. Attention is only one transformation in the path between those two objects. The remaining transformations determine where normalization acts, how information is accumulated, which tensor dominates activation memory, and how the output vocabulary is scored.

This chapter gives that path a precise, high-level form. The aim is not to rederive attention, positional encoding, normalization, or feed-forward networks in isolation. Those analyses belong to Chapters 3 and 4. Instead, the present chapter establishes the computation graph that makes later details meaningful: a sequence enters a residual stream, each block writes updates to that stream, and a final projection turns the resulting states into logits for the language-model objective of Chapter 5.

2.1 From token IDs to hidden states

Consider a batch of B token sequences, each represented to length T . Let

$$X \in \{0, \dots, |\mathcal{V}| - 1\}^{B \times T} \quad (9)$$

be the integer token-ID tensor, let d denote the model dimension, and let $E \in \mathbb{R}^{|\mathcal{V}| \times d}$ be the token-embedding matrix from Chapter 1. Embedding lookup produces a rank-three tensor

$$H^0 = E[X] \in \mathbb{R}^{B \times T \times d}. \quad (10)$$

The superscript identifies depth rather than a power. Thus H^0 is the sequence representation before the first Transformer block, while H^ℓ will denote the representation after block ℓ . If the architecture uses additive absolute positions, the input is instead $H^0 = E[X] + P$, where $P \in \mathbb{R}^{T \times d}$ is broadcast across the batch. Other positional schemes, including RoPE, modify later computations rather than this initial tensor. In every case, the central object is the same: for batch item b and position t , $H_{b,t,:}^0$ is a length- d hidden-state vector.

The distinction between IDs and states is fundamental. X is discrete and has no useful local geometry: token ID 800 is not intrinsically closer to ID 801 than to ID 7. Each slice of H^0 is continuous and trainable. Subsequent layers can mix information across positions and features without changing the sequence length. Chapter 1 established how the embedding table creates this representation; here we follow it through the complete model.

2.2 The decoder-only stack

Let L be the number of Transformer blocks. A decoder-only model applies a sequence of functions $F_1, \dots, F_L$ to the input representation:

$$H^\ell = F_\ell(H^{\ell-1}), \quad \ell \in \{1, \dots, L\}. \quad (11)$$

Each F_ℓ preserves the shape $B \times T \times d$. The word **decoder-only** refers to the causal dependency pattern within these functions: the state at position t may draw on positions at most t , but not on later tokens. It does not mean that the block processes positions one at a time during training. The full tensor is usually evaluated in parallel under a causal Attention Mask. The sequential aspect of generation emerges only when the model is used to append a new token to an existing prefix.

The original Transformer paired an encoder stack with a decoder stack for sequence-to-sequence tasks [5]. A Foundation Model used for Causal Language Modeling retains the masked, autoregressive branch and omits encoder-decoder cross-attention. This reduction creates a uniform contract: a single stream of token positions is transformed repeatedly, then scored against the vocabulary. GPT-2 is an influential example of this decoder-only pattern [3].

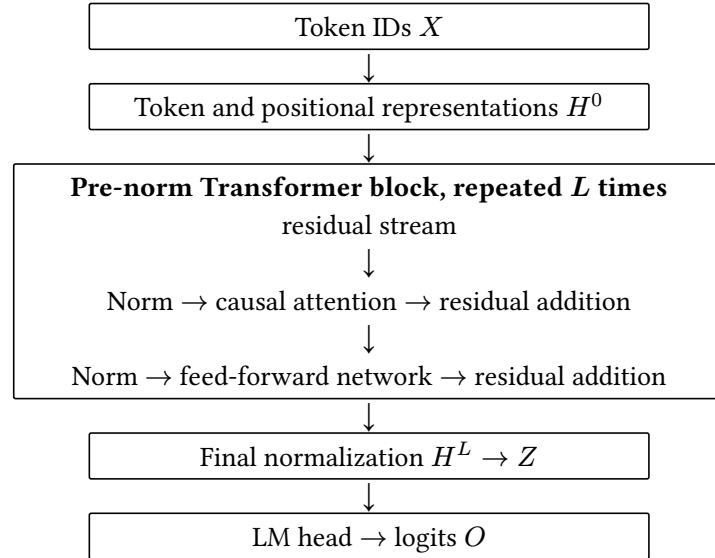


Figure 1: A high-level decoder-only Transformer computation path. Each block reads the current residual stream and writes an update to it; Chapter 3 expands causal attention and Chapter 4 expands the feed-forward and normalization components.

The diagram in Figure 1 omits internal parameter matrices on purpose. It exposes the architectural invariant that matters at this stage: every block receives a hidden-state tensor and returns a tensor of the same shape. This makes depth a controlled composition of representation updates rather than a sequence of incompatible interfaces.

2.3 Anatomy of a Transformer block

A standard decoder-only block has two learned sublayers: causal Self-Attention and a position-wise feed-forward network (FFN). Attention allows the representation at one position to use an allowed prefix of the sequence; the FFN applies the same nonlinear transformation independently at every position. Residual connections preserve a direct route around both sublayers, while normalization controls the scale and distribution of the vectors passed into them.

For a Pre-Norm block, write A_ℓ for the attention transformation, M_ℓ for the FFN transformation, and N_ℓ^a and N_ℓ^m for their respective normalizations. The two updates are

$$U^\ell = H^{\ell-1} + A_{\ell(N_\ell^a(H^{\ell-1}))}, \quad (12)$$

followed by

$$H^\ell = U^\ell + M_{\ell(N_\ell^m(U^\ell))}. \quad (13)$$

These equations describe the control flow, not an attempt to hide the details in notation. A_ℓ contains the query, key, value, output, mask, and softmax operations developed in Chapter 3. M_ℓ contains the expansion, activation or gate, and contraction developed in Chapter 4. Their outputs have the same final feature dimension d as their inputs, so they can be added to the residual stream.

The order of the two sublayers is conventional rather than mathematically necessary. The important point is that attention and the FFN have complementary roles. Attention mixes **positions**: a token representation can incorporate information from its prefix. The FFN mixes **features** within one position: it turns the resulting feature vector into a richer nonlinear update. A model needs both forms of computation to make context-sensitive predictions without treating every feature interaction as an attention operation.

2.4 Pre-Norm and Post-Norm

The original Transformer uses a Post-Norm organization, in which normalization follows each residual addition. For a generic sublayer S , that pattern is

$$y = N(x + S(x)). \quad (14)$$

By contrast, Pre-Norm applies normalization to the sublayer input and leaves the residual path itself unnormalized:

$$y = x + S(N(x)). \quad (15)$$

Neither expression is merely a rearrangement of the other. In Post-Norm, the next sublayer receives a normalized mixture of the previous stream and the current update. In Pre-Norm, the identity path from x to y remains explicit, while the learned update is driven by a normalized argument. Consequently, normalization placement changes both forward activation statistics and the paths followed by gradients.

This distinction became practically consequential as Transformer stacks grew deeper. Xiong et al. analyze the initialization behavior of Pre-Norm and Post-Norm Transformers and relate Post-Norm's output-layer gradient scale to the usefulness of warmup [6]. Modern decoder-only architectures commonly adopt a Pre-Norm family of designs; GPT-2, for example, places LayerNorm at the input of each sub-block [3]. These observations do not make Pre-Norm universally superior. They identify a stability trade-off whose interaction with optimizer choice, residual scaling, precision, and depth must be evaluated as a system. Chapter 4 returns to those mechanisms, and Chapters 7 and 8 address their training consequences.

2.5 The residual stream

The phrase **residual stream** is a useful abstraction for H^ℓ . At any depth, it is the common sequence representation carried through the network. Attention does not replace that representation; it reads a normalized view of it and adds a context-dependent update. The FFN then reads the updated stream and adds a position-wise update. Equations Equation 12 and Equation 13 therefore describe an additive computation in which information can persist across many layers while new features are written into the same width- d state.

This view has two benefits. First, it makes tensor shapes easy to audit: the residual stream remains $B \times T \times d$ from input to final normalization. Second, it discourages a misleading picture in which a Transformer repeatedly discards and reconstructs a sequence. Each block instead makes bounded modifications to a shared representation. The identity terms in the residual connections provide direct forward and backward routes, while the learned sublayers specialize in producing useful corrections. The quality of those corrections depends on the attention, FFN, and normalization choices studied later.

2.6 From final hidden states to logits

After the final block, many decoder-only models apply a final normalization N_f :

$$Z = N_{f(H^L)}, \quad Z \in R^{B \times T \times d}. \quad (16)$$

The language-model head is an affine map from each final hidden vector to one score per vocabulary item. With output matrix $W_{\text{out}} \in R^{d \times |\mathcal{V}|}$ and bias $b_{\text{out}} \in R^{|\mathcal{V}|}$,

$$O = ZW_{\text{out}} + b_{\text{out}}, \quad O \in R^{B \times T \times |\mathcal{V}|}. \quad (17)$$

The element $O_{b,t,v}$ is a logit, not yet a probability: it is the unnormalized score assigned to vocabulary item v at position t . Chapter 5 will apply a masked cross-entropy objective to these scores. At inference time, a decoding rule will convert the final position's logits into a distribution or selected next token.

When input and output embeddings are tied, the model reuses the token-embedding table and takes $W_{\text{out}} = E^T$, up to the orientation convention of the implementation. Weight tying reduces parameter count and connects the geometry used to read a token with the geometry used to score it. It is an architectural choice, not an identity required by decoder-only modeling. The original Transformer discusses shared embedding and pre-softmax weights in some configurations [5].

2.7 Tensor shapes through the model

The following table records the shape contracts most useful when implementing or debugging the high-level forward pass. Let h be the number of query heads, d_h the per-head dimension, and d_{ff} the FFN intermediate width; ordinarily $d = hd_h$.

Quantity	Shape	Role
Token IDs X	$B \times T$	Discrete input indices.
Embeddings / residual stream H^ℓ	$B \times T \times d$	Sequence representation at depth ℓ .
Queries, keys, values	$B \times T \times h \times d_h$	Head-partitioned attention inputs.
Attention output	$B \times T \times d$	Contextual update projected back to model width.
FFN intermediate	$B \times T \times d_{\text{ff}}$	Expanded position-wise representation.
Logits O	$B \times T \times \mathcal{V} $	Vocabulary scores for the language-model objective.

Table 1: Symbolic tensor shapes in a decoder-only Transformer. The exact internal layout of heads may vary by implementation, but the residual-stream and logit contracts are stable.

The table distinguishes model-width tensors from vocabulary-width tensors. Most block activations have width d , whereas logits have width $|\mathcal{V}|$. For a large vocabulary, materializing logits for every position can itself be substantial. Conversely, the residual stream persists at every block and is a major source of activation memory during training. Shape reasoning is therefore not bookkeeping after the fact; it is the first step toward a reliable memory and compute model.

2.8 A computational view

At a high level, each block contains four dense projection families associated with attention: query, key, value, and output. In a conventional full-width implementation, they contribute on the order of $4d^2$ parameters per layer. The FFN contributes roughly $2dd_{\text{ff}}$ parameters for a two-projection design, or a different constant factor for gated variants such as SwiGLU. These counts are only first-order approximations, but they already explain why changing d , L , or d_{ff} changes the parameter budget materially.

The principal sequence-length dependence arises in attention. For a length- T sequence, each position can compare against a prefix whose total pairwise work grows on the order of T^2d for dense Self-Attention. The FFN instead performs work on each position independently, scaling on the order of Tdd_{ff} . The relative bottleneck therefore changes with sequence length, width, batch size, hardware, and kernel implementation. FlashAttention, cache-aware decoding, and other inference methods will revisit the operational meaning of these expressions in Chapters 14 and 15; Chapter 17 will turn them into more complete FLOPs and memory accounting.

Activation memory has a different emphasis from parameter count. During training, backward computation requires access to selected forward intermediates or to recomputed equivalents. The residual stream alone stores BTd scalars per layer before accounting for attention and FFN internals. During autoregressive decoding, the model instead retains key and value representations for prior positions in a KV Cache. The forward equations are the same in spirit, but the dominant persistent state changes with the execution regime.

2.9 Chapter summary

A decoder-only Transformer maps token IDs to embeddings, repeatedly applies residual attention and FFN updates, normalizes the final stream, and projects each hidden state to vocabulary logits. The stack in Equation 11 supplies the architectural spine; the Pre-Norm updates in Equation 12 and

Equation 13 make the residual stream explicit; and the LM head in Equation 17 closes the path back to the discrete vocabulary.

The next two chapters unpack the sublayers that this overview deliberately treated as modules. Chapter 3 derives causal attention, its masks, head variants, and position encoding. Chapter 4 studies FFNs, normalization, residual scaling, and gradient flow. Chapter 5 then uses the logits of Equation 17 to define the Causal Language Modeling objective. Before proceeding, the reader should be able to trace every tensor in the shape contracts of Table 1 from its input contract to its output contract and identify which components scale with vocabulary size, sequence length, model width, and depth.

Chapter 3

Attention and Position Encoding

TODO: derive causal Self-Attention, attention masks, MHA / MQA / GQA, and RoPE.

Chapter 4

Feed-Forward Networks, Normalization, and Residuals

TODO: explain FFN / SwiGLU, LayerNorm / RMSNorm, initialization, residual scaling, and gradient flow.

Part II — Pretraining

Chapter 5

Language Modeling Objectives

TODO: derive Causal Language Modeling, next-token prediction, teacher forcing, cross entropy, masking, and perplexity.

Chapter 6

Pretraining Data

TODO: cover deterministic data pipelines, provenance, filtering, packing, sharding, Deduplication, quality, mixing, and curriculum.

Chapter 7

Optimization and Training Dynamics

TODO: cover AdamW, Optimizer State, schedules, Warmup, batches, Token Budget, Gradient Accumulation, Gradient Clipping, Mixed Precision, and numerical stability.

Chapter 8

Scaling, Compute, and Training Stability

TODO: cover FLOPs, Chinchilla-style scaling, compute planning, loss spikes, monitoring, Checkpoint, and recovery.

Part III — Post-training and Adaptation

Chapter 9

Supervised Fine-Tuning and Preference Data

TODO: cover instruction-response data, chat templates, loss masking, preference pairs, Reward Models, calibration, and preference-data quality.

Chapter 10

Reinforcement Learning Foundations

TODO: cover trajectories, returns, policy gradients, Actor-Critic methods, On-policy and Off-policy learning, advantages, and GAE.

Chapter 11

RLHF and PPO

TODO: derive KL-regularized policy improvement, PPO clipping, value loss, entropy, rollout/update ratios, and policy drift.

Chapter 12

DPO, GRPO, and Reasoning RL

TODO: cover DPO, GRPO, DAPO, Reward Hacking, Length Bias, Verifiable Reward, Reasoning RL, and Expert Iteration.

Chapter 13

Parameter-Efficient Fine-Tuning

TODO: cover LoRA, QLoRA, Adapters, Prefix Tuning, Prompt Tuning, quantized base weights, and deployment trade-offs.

Part IV — Inference

Chapter 14

Autoregressive Decoding and KV Cache

TODO: cover greedy decoding, Temperature, Top-k, Top-p, Beam Search, repetition controls, prefill/decode, cache layout, and KV-cache memory.

Chapter 15

Efficient Inference

TODO: cover FlashAttention, Sparse Attention, MLA, inference and KV-cache quantization, calibration, and Speculative Decoding.

Chapter 16

LLM Serving

TODO: cover PagedAttention, Continuous Batching, scheduling, queueing, vLLM, throughput, and tail latency.

Part V — LLM Systems

Chapter 17

GPU Memory, FLOPs, and Performance

TODO: cover parameters, gradients, Optimizer State, activations, GPU Memory, FLOPs, throughput, MFU, and arithmetic intensity.

Chapter 18

Distributed Training and Parallelism

TODO: cover process groups, topology, All-Reduce, All-Gather, Reduce-Scatter, Data Parallel, ZeRO, FSDP, Tensor Parallel, Sequence Parallel, Pipeline Parallel, and hybrid parallelism.

Chapter 19

Training Infrastructure and Fault Recovery

TODO: cover distributed checkpointing, resharding, reproducible data state, failure recovery, Ray, veRL, DataProto, and observability.

Part VI — Evaluation and Applications

Chapter 20

Foundation Model Evaluation

TODO: cover perplexity, Benchmarking, benchmark design, LLM-as-a-Judge, Reward Evaluation, reasoning evaluation, and data contamination.

Chapter 21

RAG, Tool Calling, and Agent Systems

TODO: cover Prompting, RAG, Tool Calling, structured outputs, Agents, LangChain-style orchestration, and Agent Evaluation.

References

- [1] R. Sennrich, B. Haddow, and A. Birch, “Neural Machine Translation of Rare Words with Subword Units,” in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Association for Computational Linguistics, 2016, pp. 1715–1725. doi: 10.18653/v1/P16-1162.
- [2] P. Gage, “A New Algorithm for Data Compression,” *C Users Journal*, vol. 12, no. 2, pp. 23–38, 1994, [Online]. Available: https://www.derczynski.com/papers/archive/BPE_Gage.pdf
- [3] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners,” technical report, 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf
- [4] T. Kudo and J. Richardson, “SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing,” in *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, Association for Computational Linguistics, 2018, pp. 66–71. doi: 10.18653/v1/D18-2012.
- [5] A. Vaswani *et al.*, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems 30*, 2017, pp. 5998–6008.
- [6] R. Xiong *et al.*, “On Layer Normalization in the Transformer Architecture,” in *Proceedings of the 37th International Conference on Machine Learning*, in *Proceedings of Machine Learning Research*, vol. 119. PMLR, 2020, pp. 10524–10533.